Sistemas Digitales · Cuaderno de Resúmenes

Nombre: **Miguel Alejandro Armas Ordóñez**

Unidad 1 · Microcontroladores para sistemas embebidos | Semanas 1–4

Bloque	Temas de tus notas	Páginas del cuaderno
01 · Fundamentos	Sistemas embebidos, CPU, buses, memorias, periféricos	1–4
02 · Bits y registros	Binario, hexadecimal, operaciones, máscaras, DDR/PORT/PIN	4–7
03 · ESP32	AVR vs. ESP32, placa, GPIO y tipos de datos	8–9
04 · Tiempo	Temporizadores, prescaler, resolución, alarma e ISR	10–11

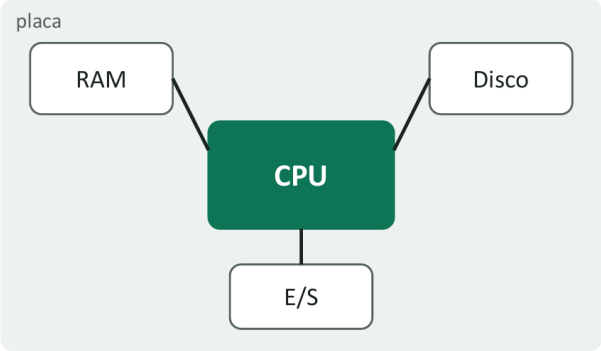
01 · Del computador al microcontrolador

Un sistema embebido es básicamente un equipo que lleva un microcontrolador para hacer una tarea concreta. Por ejemplo, una alarma no necesita funcionar como una computadora completa: solo debe leer sus sensores y responder cuando ocurre algo.

En el microprocesador, la cpu va en el chip, pero la memoria RAM, el almacenamiento y periféricos están fuera, su uso de es de propósito general como PC.

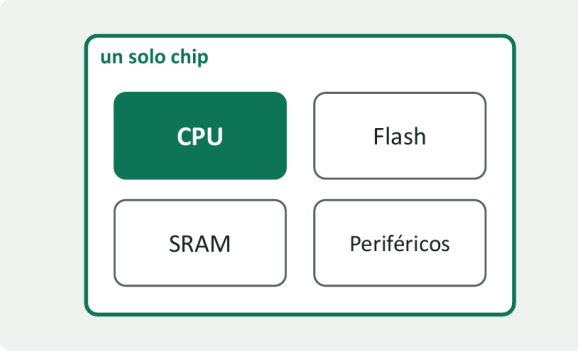
El microcontrolador, integra el CPU, memoria y periféricos en un solo chip, se usa en sistemas embebidos, sensores, alarmas.

Microprocesador



La CPU sola; memoria y E/S en otros chips. PC, laptop: propósito general.

Microcontrolador



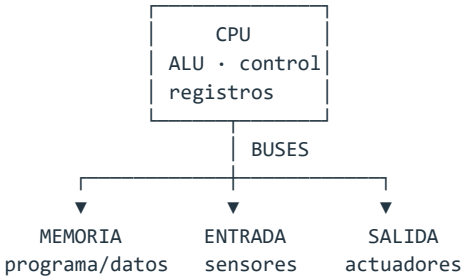
Todo en un chip: barato y de bajo consumo. Lavadora, sensor, alarma: una tarea.

Microprocesador	Microcontrolador
La CPU necesita memoria y periféricos externos para formar un sistema.	Integra CPU, memoria y periféricos en un mismo chip.
Se orienta a propósito general.	Se orienta a controlar tareas específicas, usualmente con bajo consumo.

Elementos fundamentales

- CPU: se encarga de ejecutar el programa. Dentro están la unidad de control, la ALU y los registros.
- Memoria: aquí se guarda tanto el programa como la información con la que está trabajando.
- Periféricos: son los bloques que permiten interactuar con el exterior: GPIO, temporizadores, ADC, DAC, PWM y puertos de comunicación.
- Buses: conectan la CPU con las memorias y los periféricos. No todos transportan lo mismo.

Esquema de referencia



Componente	Función
CPU	Procesa instrucciones y controla operaciones
ALU	Operaciones aritméticas y lógicas
Registros	Almacenamiento interno rápido
Memoria	Guarda código y datos
Periféricos	Comunicación con el exterior

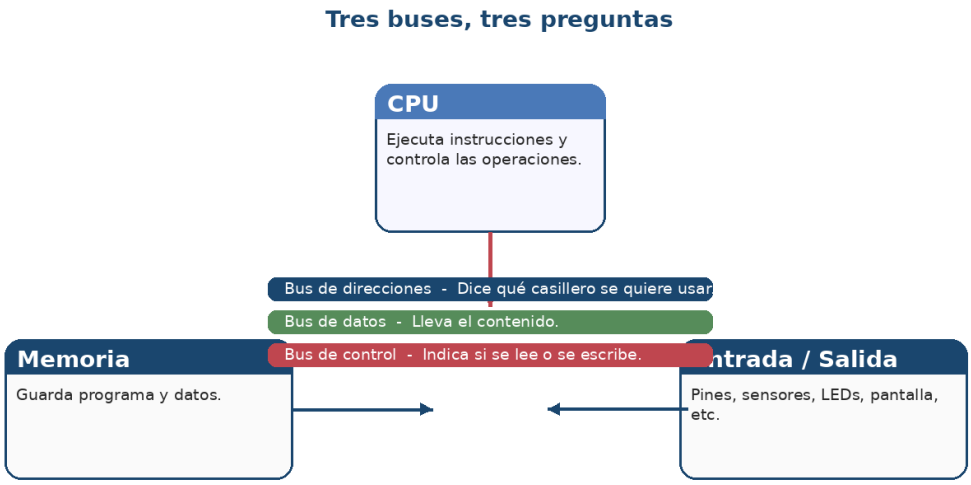
Una instrucción pasa por tres etapas: se busca, se interpreta y se ejecuta. Cuando termina, la CPU continúa con la siguiente.

La CPU trabaja por ciclos

La CPU repite el proceso de buscar → decodificar → ejecutar. El reloj marca el ritmo; eso no significa que cada instrucción tarde exactamente un ciclo.

Componente	Función
ALU	Operaciones aritméticas y lógicas.
Unidad de control	Interpreta instrucciones y coordina su ejecución.
Registros	Almacenamiento interno muy rápido.
Contador de programa (PC)	Dirección de la siguiente instrucción.

02 · Buses y memorias



Bus	Pregunta que resuelve	Contenido
Direcciones	¿Dónde?	Dirección de memoria o periférico.
Datos	¿Qué?	Bits que se leen o escriben.
Control	¿Qué operación?	Señales de lectura y escritura.

Para escribir un dato, la CPU primero señala la dirección, después coloca el valor y activa la escritura. Para leer, señala la dirección y recibe el dato. Es la misma idea que usamos cuando trabajamos con registros.

Harvard frente a von Neumann

En von Neumann se comparte la vía para instrucciones y datos. En Harvard se separan. El ATmega328P, que usamos como ejemplo, tiene arquitectura Harvard.

Flash, SRAM y EEPROM

Memoria	Se conserva sin energía	Uso en clase
Flash	Sí	Programa y constantes.
SRAM	No	Variables durante la ejecución.
EEPROM	Sí	Configuraciones y datos que deben persistir.

Relaciones principales

Bus	Pregunta	Contenido
Direcciones	¿Dónde?	Dirección de memoria o registro
Datos	¿Qué?	Valor que se lee o escribe
Control	¿Qué operación?	Leer / escribir

```
CPU ──direcciones──► MEMORIA / PERIFÉRICO
CPU ◄─────datos─────► MEMORIA / PERIFÉRICO
CPU ─────control─────► leer / escribir
```

Memorias y arquitecturas

Memoria	¿Se conserva al apagar?	Uso
Flash	Sí	Programa

SRAM	No	Variables temporales
EEPROM	Sí	Datos persistentes

```
Von Neumann: CPU ⇄ [programa + datos] (bus compartido)
Harvard:     CPU ⇄ [programa]  y  CPU ⇄ [datos]
```

Cuánto espacio ocupa un arreglo

El tamaño de un arreglo se calcula multiplicando el número de elementos por los bytes de cada elemento. En AVR, int suele ocupar 2 bytes: int[2000] necesitaría 4 000 bytes y no cabría en los 2 KB de SRAM del Arduino Uno.

03 · Periféricos, GPIO y niveles lógicos

Los GPIO son pines de entrada y salida. Sirven para leer señales como las de un pulsador y para manejar salidas como un LED. Además, el microcontrolador tiene periféricos especializados, por ejemplo ADC, timers, PWM, UART, SPI e I^2C.

Periférico	Para qué sirve
GPIO	Leer o escribir un estado digital.
Timer	Medir intervalos y producir avisos por hardware.
ADC / DAC	Conversión entre señales analógicas y valores digitales, en sentidos opuestos.
PWM	Señal de pulsos con ciclo de trabajo controlable.
UART / I^2C / SPI	Comunicación con otros dispositivos.

Un 0 y un 1 son niveles lógicos, no números aislados. Como referencia, el Arduino Uno trabaja a 5 V y el ESP32 a 3,3 V. Esto importa al conectar componentes: no se debe mandar directamente una señal de 5 V a un GPIO del ESP32.

Periféricos

Grupo	Elementos de los apuntes	Uso básico
Digital	GPIO	Entrada o salida: 0 / 1
Temporización	Timers / PWM	Contar tiempo / modular pulsos
Conversión	ADC / DAC	Analógico ↔ digital
Comunicación	UART / I^2C / SPI	Intercambiar información

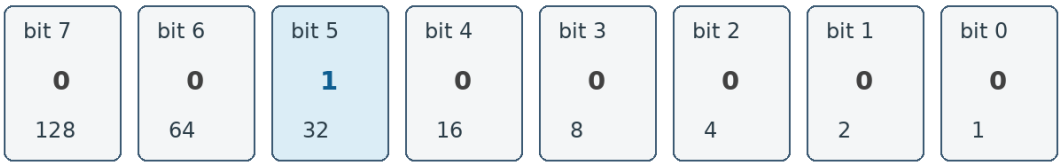
```
Sensor / pulsador → GPIO → CPU → GPIO → LED / actuador
```

Referencias de clase: nivel bajo ≈ 0 V. Nivel alto ≈ 5 V (Arduino Uno) o ≈ 3,3 V (ESP32). Los umbrales reales de lectura dependen del dispositivo.

04 · Bits, bytes, binario y hexadecimal

Un byte tiene 8 bits. Para convertir de binario a decimal, sumo el valor de cada posición que tiene un 1. Los pesos van de 128 a 1, empezando por el bit 7.

Ejemplo: 0010 0000 = 32 = 0x20



Valor = suma de los pesos cuya casilla vale 1

Representación	Ejemplo	Lectura
Binario	0010 0000	32 en decimal
Hexadecimal	0x20	32 en decimal
Decimal	255	0xFF = 1111 1111

Conversión entre bases

```
Posición:  7   6   5   4   3   2   1   0
Peso:    128  64  32  16   8   4   2   1
32:        0   0   1   0   0   0   0   0
5:         0   0   0   0   0   1   0   1
```

Decimal	Binario (8 bits)	Hexadecimal
5	0000 0101	0x05
32	0010 0000	0x20
129	1000 0001	0x81
255	1111 1111	0xFF

$0x81 = 1000\ 0001_2 = 128 + 1 = 129_{10}$
$1 << 5 = 0010\ 0000_2 = 0x20$

Cuatro bits forman un dígito hexadecimal

Un dígito hexadecimal representa 4 bits. Por eso 1100 1000 se separa en 1100 (C) y 1000 (8), es decir, 0xC8. También: 0xFF = 255 y 0x08 = 8.

05 · Operaciones bit a bit y máscaras

Operador	Nombre	Regla esencial
&	AND	1 solamente si ambos bits son 1.
|	OR	1 si al menos uno es 1.
^	XOR	1 si los bits son distintos.
~	NOT	Invierte los bits.
<<	Desplazamiento izquierdo	Desplaza los bits hacia la izquierda.

Una máscara sirve para trabajar con un bit en particular. Si escribo 1 << 5, el 1 se mueve cinco posiciones y obtengo 0010 0000, que también se escribe 0x20.

```
REG |=  (1 << n);  // poner el bit n en 1
REG &= ~(1 << n);  // poner el bit n en 0
REG ^=  (1 << n);  // invertir el bit n
if (REG & (1 << n)) { /* comprobar bit */ }
```

Operaciones resueltas

Acción anotada	Expresión en C	Por qué funciona
Poner bit n en 1	REG |= (1U << n);	OR con 1 enciende el bit
Poner bit n en 0	REG &= ~(1U << n);	AND con 0 borra el bit
Invertir bit n	REG ^= (1U << n);	XOR con 1 invierte el bit
Consultar bit n	if (REG & (1U << n))	AND conserva solo el bit consultado

```
          0010 1010
OR        1000 0000
          ─────────
          1010 1010

          0010 1010
AND       1101 1111
          ─────────
          0000 1010
```

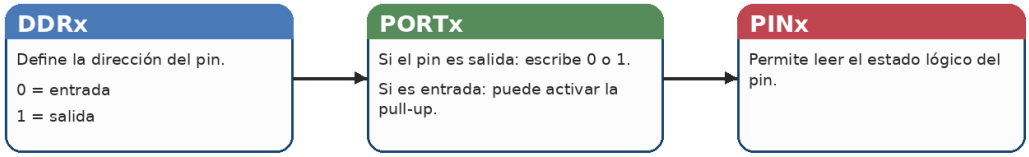
Operadores: AND (&), OR (|), XOR (^) y NOT (~). Con una máscara se puede encender, apagar, invertir o consultar un bit.

La idea es no alterar los demás bits del registro. Para poner uno en 1 uso OR; para borrarlo, AND con la máscara negada; y para invertirlo, XOR.

06 · Registros AVR: DDR, PORT y PIN

En AVR hay tres registros que se repiten bastante: DDR indica si el pin entra o sale, PORT escribe el valor o activa la pull-up y PIN permite leer lo que llega desde afuera.

Relación entre DDR, PORT y PIN



Lectura rápida

DDRx=0 y PORTx=0	Entrada flotante
DDRx=0 y PORTx=1	Entrada con pull-up
DDRx=1 y PORTx=0	Salida en bajo
DDRx=1 y PORTx=1	Salida en alto

Registro	Función	Interpretación
DDRx	Dirección	0 = entrada; 1 = salida.
PORTx	Salida / pull-up	Escribe el nivel de salida; en entrada puede habilitar pull-up.
PINx	Entrada	Permite consultar el nivel leído.

```
DDRB  |=  (1 << PB5);  // PB5 como salida
PORTB |=  (1 << PB5);  // salida alta
PORTB &= ~(1 << PB5);  // salida baja
```

```
if (PIND & (1 << PD2)) { /* leer PD2 */ }
```

Registros AVR · Puertos y ensamblador

Registro	Uso	Ejemplo
DDRB	Configura dirección: 1 salida / 0 entrada	DDRB |= (1 << PB5);
PORTB	Escribe nivel de salida; pull-up si es entrada	PORTB |= (1 << PB5);
PINB	Lee el nivel de entrada	PINB & (1 << PB5)

Instrucción	Función anotada
LDI	Carga un valor inmediato
ADD	Suma registros
SBI	Pone un bit en 1
CBI	Pone un bit en 0
RJMP	Salto relativo

```
DDRB5 = 0 → entrada          DDRB5 = 1 → salida
PORTB5 = 0 → nivel bajo      PORTB5 = 1 → nivel alto
```

Instrucciones AVR anotadas

Algunas instrucciones de AVR que aparecen en clase: LDI carga un número en un registro; ADD suma; SBI pone un bit en 1; CBI lo pone en 0; RJMP salta a otra parte del programa. Son operaciones bastante directas.

07 · Del AVR al ESP32

Característica	ATmega328P (Uno)	ESP32-WROOM-32
Arquitectura	8 bits	32 bits
Procesamiento	1 núcleo, 16 MHz	2 núcleos, hasta 240 MHz
SRAM	2 KB	520 KB
Flash	32 KB en chip	4 MB externa en el módulo
Lógica	5 V	3,3 V
Asignación periféricos	Más fija	Configurable (GPIO matrix)

La lógica de entradas y salidas sigue siendo parecida al cambiar del AVR al ESP32. Lo que cambia es la forma de configurar el pin y los nombres de los registros. En el ESP32 también está la matriz GPIO, que permite asignar distintas funciones a diferentes pines.

Propósito	AVR	ESP32 (GPIO 0–31)
Habilitar salida	DDRB	GPIO_ENABLE_REG
Poner o quitar salida	PORTB	GPIO_OUT_W1TS_REG / GPIO_OUT_W1TC_REG
Leer pin	PINB	GPIO_IN_REG

AVR y ESP32 · Correspondencia de registros

Función	AVR	ESP32
Dirección	DDRB	GPIO_ENABLE_REG
Salida: poner a 1	PORTB	GPIO_OUT_W1TS_REG
Salida: poner a 0	PORTB	GPIO_OUT_W1TC_REG
Entrada	PINB	GPIO_IN_REG
Pull-up	PORT (modo entrada)	IO_MUX por pin

```
USB → USB/serie → ESP32-WROOM-32 → GPIO / periféricos
                       |          |
                  regulador    Flash externa
```

No hay que confundir el nombre GPIO con la posición del pin en la placa. Por ejemplo, GPIO18 no significa necesariamente “pin físico número 18”. Lo mejor es revisar la serigrafía y el diagrama de la DevKit.

W1TS y W1TC

En el ESP32, W1TS permite poner en 1 los bits señalados y W1TC permite borrarlos. En ambos casos, escribir 0 no cambia ese bit. Así se evita leer todo el registro, modificarlo y volverlo a escribir, lo cual puede causar problemas si otro proceso lo cambia entre medias.

08 · Pines ESP32 y tipos de datos

Los GPIO 34 a 39 del ESP32 clásico solo sirven como entrada y no tienen resistencia pull-up interna. Es un detalle fácil de pasar por alto cuando se conecta un botón o un sensor.

Para las prácticas se revisaron GPIO de uso común como 16, 17, 18 y 19. De todos modos, conviene verificar el pinout antes de conectar un LED, un pulsador o un periférico.

Tipos con tamaño explícito

Tipo	Tamaño	Rango
int8_t	8 bits	−128 a 127
uint8_t	8 bits	0 a 255
int16_t	16 bits	−32 768 a 32 767
uint16_t	16 bits	0 a 65 535
int32_t	32 bits	aprox. ±2,1 mil millones
uint32_t	32 bits	0 a 4 294 967 295

En el Arduino Uno, int ocupa 2 bytes; en el ESP32 ocupa 4. Para evitar confusiones con el tamaño, es preferible escribir uint8_t, int16_t, uint32_t, etc., cuando los bits sí importan.

Pines y tipos de datos

Categoría	GPIO
LED (ejemplos)	16, 17, 18, 19
Pulsador (ejemplo)	21
Evitar para uso normal	6 a 11 (flash)
Solo entrada	34, 35, 36, 39
Arranque: precaución	0, 2, 5, 12, 15

Tipo	Bytes en ESP32	Rango o nota
char	1	Depende de signed/unsigned
short	2	16 bits
int	4	32 bits
long	4	32 bits
float	4	Coma flotante
double	8	Coma flotante
uint8_t	1	0–255
int8_t	1	−128 a 127
uint16_t	2	0–65535
uint32_t	4	0–4294967295

Dato importante: una operación con uint8_t no siempre se calcula en 8 bits. El compilador puede convertir los operandos a int antes de hacer la cuenta.

Desbordamiento y promoción

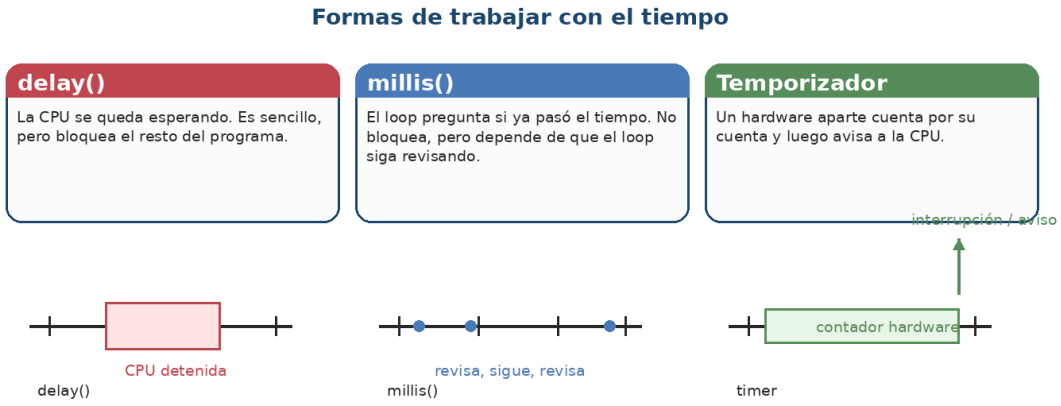
Ejemplo: si sumo 200 + 100, el resultado es 300. Pero uint8_t solo llega hasta 255: al guardar 300 queda 44. Con enteros sin signo el contador da la vuelta; con signo no se debe confiar en un desbordamiento.

El pulsador no cambia limpiamente

Un botón físico puede producir varios cambios muy rápidos al presionarlo. A eso se le llama rebote. En la práctica se espera a que la señal esté estable (por ejemplo, 30 ms) y después se toma el cambio como una sola pulsación.

09 · Temporizadores por hardware

El temporizador no es una variable que el loop aumenta manualmente. Es un circuito del microcontrolador que va contando pulsos de reloj y puede generar un aviso cuando alcanza el valor configurado.



Reloj, prescaler y tick

El prescaler divide la frecuencia de entrada del temporizador. Si llegan menos pulsos por segundo, cada tick representa más tiempo. La resolución es justamente el tiempo que dura un tick.

```
frecuencia_timer = frecuencia_reloj / prescaler
resolución = 1 / frecuencia_timer
tiempo = valor_alarma × resolución
valor_alarma = tiempo_deseado / resolución
```

Por ejemplo, con una frecuencia de 10 000 ticks/s, cada tick dura 0,1 ms. Si la alarma está en 2 500 ticks, el aviso llega después de 250 ms.

Funcionamiento del temporizador

```
Reloj APB → prescaler → pulso (tick) → contador → alarma → ISR
                                0 → 1 → 2 → 3 → … → N
```

Concepto	Definición del apunte
Contador	Registro que incrementa con cada pulso de reloj
Prescaler	Divisor de la frecuencia de entrada
Resolución	Duración de cada tick
Alarma	Valor de cuenta que provoca un aviso
Autorecarga	Reinicia la cuenta para repetir intervalos

```
Frecuencia del contador = frecuencia del reloj / prescaler
Resolución = 1 / frecuencia del contador
Tiempo de alarma = valor de alarma × resolución
```

Ejemplo rápido con prescaler

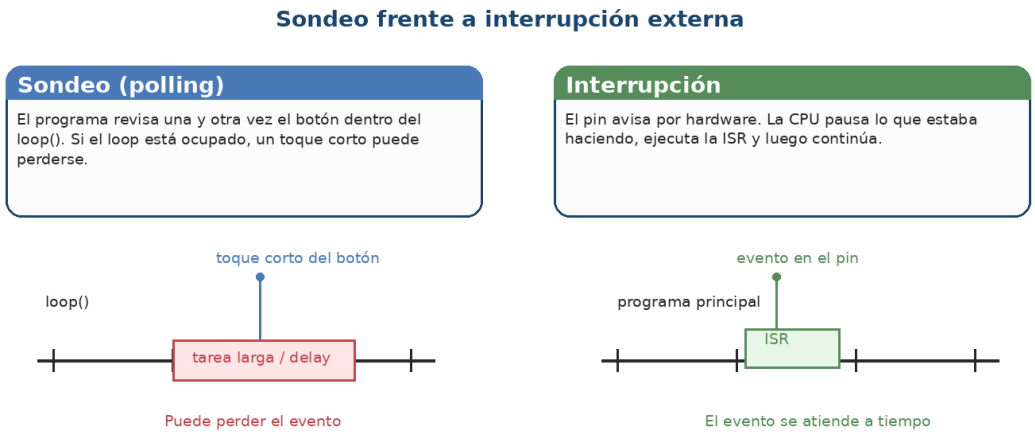
Si el reloj es de 80 MHz y se usa un divisor de 80, el contador queda en 1 MHz. Cada tick dura 1 µs; por tanto, 500 000 ticks equivalen a medio segundo. Este cálculo sirve para escoger el valor de alarma (S3, tema de prescaler).

Autorecarga e interrupción

Con autorrecarga, al alcanzar la alarma el contador se reinicia según la configuración y vuelve a contar. No es lo mismo que el desbordamiento por llegar al máximo del registro.

10 · ISR y organización del programa

ISR significa Interrupt Service Routine: es la función que atiende el aviso de una interrupción. Debe ser breve; lo mejor es que registre el evento y que el resto del trabajo lo haga el programa principal.



Para una interrupción externa se configura el GPIO y el flanco que debe detectarse. Con un botón conectado a GND y pull-up, normalmente FALLING corresponde a presionar y RISING a soltar.

Tres formas de manejar el tiempo

Técnica	Cómo funciona	Consecuencia práctica
delay()	Espera bloqueando el flujo del programa.	No ejecuta el resto de loop() durante la espera.
millis()	El programa revisa cuánto tiempo ha transcurrido.	Evita una espera bloqueante, pero depende de revisar a tiempo.
Timer por hardware	El periférico cuenta y genera un aviso.	Puede señalar vencimientos sin sondeo constante.

Ejemplo de cálculo

```
Frecuencia de ticks = 10 000 ticks/s
Resolución = 1 / 10 000 s = 0,0001 s = 0,1 ms
Alarma = 2 500 ticks
Tiempo = 2 500 × 0,1 ms = 250 ms = 0,25 s
```

Elemento	Apunte digitalizado
TIM G0	Timer 0 y Timer 1
TIM G1	Timer 2 y Timer 3
ISR	Interrupt Service Routine: función al ocurrir la interrupción
Callback	Función llamada cuando ocurre un evento

